Spectral Scoring with Attention-guided Propagation for Agentic Failure Attribution

Anonymous authors
Paper under double-blind review

Abstract

Failure attribution for LLM agents aims to identify the decisive step responsible for task failure, which is critical for debugging and improving agentic systems. Existing methods rely on costly LLM prompting, step-level error annotations, or verified successful trajectories that lack failure signals for assistance. In this paper, we study failure attribution from step-unlabeled failure trajectories, where each trajectory is known to have failed but its decisive step is unknown. We propose Soap (Spectral scOring with Attention-guided Propagation), a novel framework requiring neither step-level training labels nor a separate success-only reference set. Soap first constructs a spectral reference space from pooled step representations and assigns each step a base error score according to its projection behavior. Importantly, later steps may exhibit stronger failure signals because they inherit an earlier mistake, causing independent scorers to favor downstream consequences over the original error. To address this downstream contamination, Soap derives soft step dependencies from attention patterns and propagates downstream error evidence toward its likely upstream sources. Extensive experiments show that Soap consistently improves attribution performance over competitive baselines.

1 Introduction

When an LLM-based agent fails on a long-horizon task, identifying where the failure began is essential for debugging, auditing, and improving the agent. An agent trajectory may contain dozens or even hundreds of reasoning, communication, and tool-use steps, while only one early decision introduces the error that ultimately causes task failure. Manually tracing such trajectories is prohibitively expensive and prone to inconsistency, especially as agents are increasingly deployed for software engineering, scientific discovery, and other complex tasks (Yao et al., 2023; Wang et al., 2024). This motivates the problem of failure attribution: given a failed trajectory, identify the decisive step that introduced the failure (Zhang et al., 2025c).

Failure attribution is fundamentally challenging. Existing methods typically rely either on prompting powerful LLM judges, which incurs substantial inference cost and can be unreliable over long trajectories (Zhang et al., 2025c; Banerjee et al., 2025; Wang et al., 2026; Rafi et al., 2026; Yu et al., 2025), or on post-training attribution models using failure trajectories with step-level annotations (Zhang et al., 2025a;b). Such annotations are costly and often ambiguous, since identifying the decisive step may require substantial domain expertise. OAT (Yeh et al., 2026) alleviates this burden by learning the latent dynamics of successful trajectories and detecting deviations from the learned flow. However, it has no access to failure trajectories during training, and therefore models only what successful behavior looks like. Moreover, its success-only reference set must be verified and may become stale as agents and task distributions evolve. This motivates a complementary question:

Can failed trajectories themselves enable failure attribution without step-level annotations?

Failed trajectories provide a complementary source of supervision. In many agentic systems, whether a task has failed can be determined from execution outcomes or task evaluators,

while identifying the step responsible for that failure remains costly. We therefore consider a collection of step-unlabeled failure trajectories, i.e., every trajectory is known to have failed, but no step is annotated as decisive or non-decisive. Each trajectory consequently contains an unlabeled mixture of ordinary steps and the decisive error. Unlike OAT (Yeh et al., 2026), such a failure-conditioned dataset directly exposes the behaviors that arise when agents fail. Yet harnessing this information is non-trivial: without step-level membership labels, the learner must extract a meaningful failure signal without knowing which step introduced the failure. The first challenge is therefore to identify decisive-error evidence from a step-level unlabeled mixture within failed trajectories.

A second challenge is that the step exhibiting the strongest failure signal need not be the step that introduced the failure. Agent trajectories are autoregressive because each step is generated from preceding observations and actions. Once an error occurs, later steps operate on a corrupted context and may produce failed tool calls, contradictory reasoning, or repeated recovery attempts. These downstream consequences can be more conspicuous than the original error. For example, a plausible but incorrect tool argument may trigger a sequence of visible error messages and unsuccessful retries, causing a conventional per-step scorer to select the final retry rather than the initial incorrect action. We refer to this phenomenon as downstream contamination (Figure 1). Recent methods reconstruct dependencies through LLM prompting or specialized training, then perform graph search or backtracking (Rafi et al., 2026; Wang et al., 2026; Zhang et al., 2025b), which introduce substantial inference or training overhead. This motivates a lightweight alternative that extracts soft dependencies from the model to separate root causes from downstream effects.

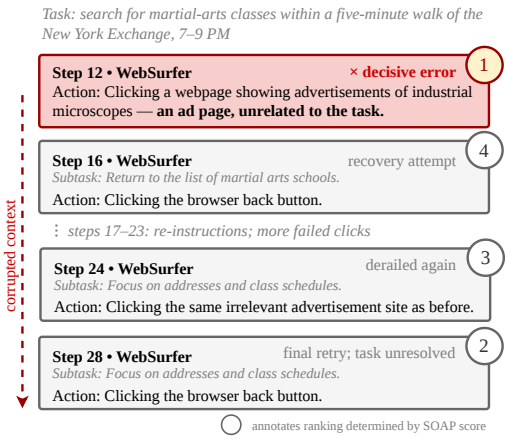


Figure 1: Downstream contamination in a failed Who&When trajectory. An early wrong click (step 12) lands the browser on an irrelevant page; later steps are spent on recovery attempts that derail again.

In this paper, we propose Soap (Spectral scOring with Attention-guided Propagation), a framework for decisive-error localization from step-unlabeled failure trajectories. At a high level, Soap consists of two components. First, it constructs a spectral reference space from step representations extracted from the failure trajectories. Each step receives a base error score according to its projection onto a selected band of singular directions. This produces a step-level signal without requiring decisive-error annotations or a separate collection of verified successful trajectories. Second, Soap addresses downstream contamination through dependency-aware rescoring. We derive a soft dependency graph from models' attention patterns, where each edge approximates how strongly a later step draws on an earlier one. Rather than treating downstream failure signals as independent and competing evidence, Soap propagates them backward toward the preceding steps on which they depend. An earlier step is therefore promoted when anomalous downstream behavior is systematically built upon it.

We evaluate Soap on [TODO: benchmarks] using [TODO: agent systems and proxy models]. Across [TODO: number] evaluation settings, Soap consistently improves decisive-step localization over different competitive baselines. In particular, Soap improves [TODO: primary metric] by [TODO: result] over the strongest baseline on [TODO: benchmark]. Ablation studies further show that both spectral step scoring and attention-guided rescoring contribute to the improvements, which cannot be explained by simply favoring earlier steps or uniformly aggregating downstream scores. [TODO: see my comments in the exp section.] Our main contributions are summarized as follows:

- We formulate decisive-error localization from step-unlabeled trajectories within failed trajectories. This setting exploits trajectory-level failure signals while requiring neither decisive-error annotations nor a carefully verified successful trajectories.
- We identify downstream contamination as a central obstacle to decisive-error localization, i.e., later steps may exhibit stronger failure signals because they inherit an earlier mistake, causing independent per-step scorers to systematically localize failures too late.
- We propose Soap, a novel framework performing spectral step scoring with attention-derived dependency propagation. Extensive experiments and ablations on [TODO: benchmarks] demonstrate when and why Soap improves attribution. The results provide insights into leveraging step-unlabeled failure trajectories for agentic failure attribution.

2 Problem Setup

We formalize the task of localizing the decisive error in a failed agent trajectory.

Agent trajectory. Let x denote a task description and let $\tau = (s_1, \ldots, s_T)$ be the resulting trajectory of a multi-agent system. Each step s_t contains the action or message produced at turn t and is associated with an agent $a_t \in \{1, \ldots, K\}$. For evaluation, a failed trajectory is paired with a gold decisive-error index $t^\star \in \{1, \ldots, T\}$, corresponding to the earliest step at which the error responsible for the eventual task failure is introduced. The responsible agent is therefore $a_{t^\star}$.

Our framework extracts step representations and attention patterns using a frozen language model $\mathcal{M}$. When the model that generates the trajectory exposes its internal states, $\mathcal{M}$ can be the generating model itself. Otherwise, $\mathcal{M}$ can be a separate open-weight proxy model, allowing the framework to analyze trajectories produced by proprietary agents whose internals are inaccessible. In either case, $\mathcal{M}$ is used without fine-tuning on failure-attribution data.

Failure attribution. Given the task description x and a failed trajectory τ, the goal of failure attribution is to estimate the decisive-error step (Zhang et al., 2025c). The learner predicts an index $\hat{t}$ as an estimator of $t^\star$, with the corresponding responsible-agent prediction $\hat{a} = a_{\hat{t}}$.

Rather than predicting the decisive-error index directly, we assign each step a scalar attribution score $\widetilde{S}(s_t) \in \mathbb{R}$, where a larger value indicates that the step is more likely to have introduced the failure. The top-1 prediction is $\hat{t} = \arg\max_{t \in \{1,\ldots,T\}} \widetilde{S}(s_t)$. Sorting the scores further produces a ranked list of candidate steps for failure triage. We evaluate both top-1 localization and ranked-set performance in §4.

Step-unlabeled failure trajectories. In addition to the test trajectory, we assume access to a reference collection of failed trajectories $\mathcal{D}_{\text{fail}} = \{\tau^{(n)}\}_{n=1}^{N}, \tau^{(n)} = (s_1^{(n)}, \ldots, s_{T_n}^{(n)})$. The task-level outcome of each trajectory is known: every trajectory in $\mathcal{D}_{\text{fail}}$ ends in failure. However, no step-level annotation is available to identify which step introduced the failure or which agent was responsible. We therefore model the pooled steps in $\mathcal{D}_{\text{fail}}$ as an unlabeled contamination mixture (Huber, 1964)

$$P_{\text{fail}} = (1 - \pi) P_{\text{ord}} + \pi P_{\text{err}}, \tag{1}$$

where P_{ord} denotes the marginal distribution of ordinary steps, P_{err} denotes that of decisive-error steps, and $\pi \in (0, 1)$ is an unknown contamination proportion. Equivalently, each trajectory has an unobserved error set $\mathcal{E}^{(n)} \subseteq \{1, \ldots, T_n\}$, typically containing a single decisive step, such that $s_t^{(n)} \sim P_{\text{err}}$ when $t \in \mathcal{E}^{(n)}$ and $s_t^{(n)} \sim P_{\text{ord}}$ otherwise. Eqn. 1 characterizes the marginal composition of the pooled steps; it does not assume that steps within a trajectory are temporally independent.

Our method uses $\mathcal{D}_{\text{fail}}$ only to construct the reference representation space, without observing $\mathcal{E}^{(n)}$ or any responsible-agent annotations. Gold step-level labels are used only for evaluation and, when applicable, hyperparameter selection on a disjoint validation split.

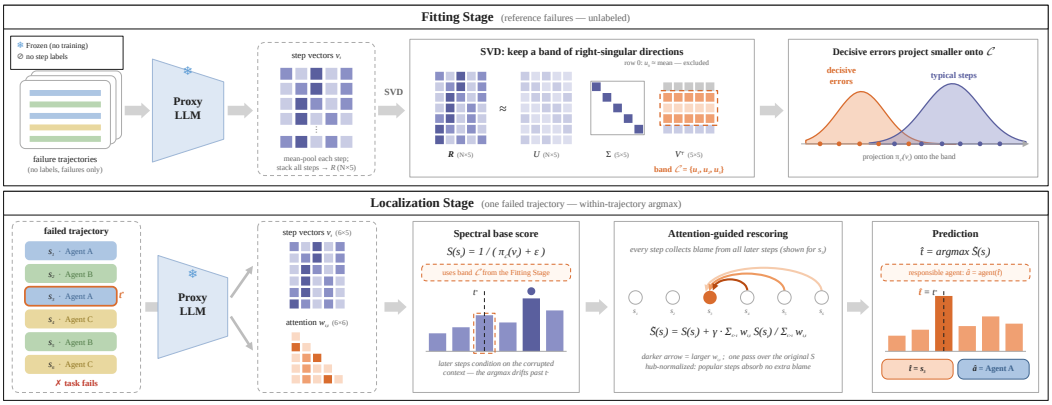


Figure 2: Overview of Soap. Fitting stage: a frozen proxy model $\mathcal{M}$ encodes every step of the unlabeled reference failures $\mathcal{D}_{\text{fail}}$ into a vector v_t; the stacked matrix R is factorized by SVD, and Soap keeps a contiguous band $\mathcal{C}$ of right-singular directions (here $\{u_1, u_2, u_3\}$; the leading direction, aligned with the mean step, is excluded). Ordinary steps project strongly onto this common mode, decisive errors weakly. Localization stage: for one failed trajectory, the same frozen proxy yields the step vectors v_t and the predecessor-attention weights $w_{i,t}$. The spectral base score $S(s_t) = 1/(\pi_{\mathcal{C}}(v_t) + \epsilon)$ flags anomalous steps, but its argmax tends to land after the decisive error $t^\star$ because every later step conditions on the corrupted context. Rescoring routes each step's score back to the predecessors it depends on (darker arrow = larger $w_{i,t}$); the argmax of the rescored $\widetilde{S}$ moves onto $t^\star$, and the responsible agent follows as $\hat{a} = a_{\hat{t}}$. Matrices, scores, and weights are illustrative. No step labels, fine-tuning, or LLM prompting is involved.

3 Soap: Spectral Scoring with Attention-Guided Propagation

In this section, we introduce Soap, a framework for localizing the decisive error from step-unlabeled failure trajectories. At a high level, Soap consists of two components. First, we construct a spectral reference space from the step representations in $\mathcal{D}_{\text{fail}}$ and derive a base error score for each step (§3.1). Second, we estimate soft dependencies between steps from the attention patterns of the model and propagate downstream error evidence toward the earlier steps on which it depends (§3.2). The resulting score $\widetilde{S}(s_t)$ estimates how likely step s_t is to have introduced the failure.

3.1 Spectral step scoring from unlabeled failure trajectories

The reference collection $\mathcal{D}_{\text{fail}}$ contains many ordinary steps but only a small number of decisive errors within each trajectory. Consequently, the representations from LLM $\mathcal{M}$ are dominated by recurring patterns of ordinary agent behavior, while decisive-error steps form a sparse, unlabeled component. Our key idea is to characterize this representation space through spectral decomposition and score each step according to its alignment with selected spectral directions.

For a trajectory $\tau = (s_1, \ldots, s_T)$, we encode each step in the context in which it was produced. Specifically, for step s_t, we provide the model $\mathcal{M}$ with the task description and the trajectory context $z_t = [x; s_1; \ldots; s_t]$. and then extract its d-dimensional step representation v_t. [Finn: refer details, e.g., context cutoff, in the appendix.]

Figure 2 summarizes the method.

Spectral reference space. We apply the same representation extraction procedure to every step in the reference collection $\mathcal{D}_{\text{fail}}$. Let $M = \sum_{n=1}^{N} T_n$ be the total number of reference steps. We stack their representations into $R = [(v_t^{(n)})^\top]_{n\in[N],\, t\in[T_n]} \in \mathbb{R}^{M\times d}$ where $v_t^{(n)}$ is representation of the t-th step in the n-th sequence or trajectory. We then compute its singular value decomposition

$$R = U\Sigma V^\top. \tag{2}$$

The columns of V describe orthogonal directions of variation among the unlabeled steps, ordered by decreasing singular value.

Importantly, the reference set R is dominated by ordinary steps because each failure trajectory typically contains only a small number of decisive errors. Consequently, the leading singular directions may primarily encode frequent shared structure, such as response style, trajectory format, and common tool-use patterns. Conversely, components at the extreme tail may contain irrelevant variation. We therefore do not assume that the most informative signal must lie in the leading components. Instead, we consider a contiguous spectral band to represent the decisive-error information $\mathcal{C} = \{c_{\text{begin}}, \ldots, c_{\text{end}} - 1\}$, whose boundaries are selected as described in §4.

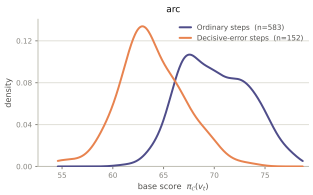


Figure 3: Base-score distribution on a CORRECT-Error test split. Decisive-error steps concentrate at smaller projection magnitudes than ordinary steps.

Spectral base score. For a step representation v_t, we measure its average squared projection onto the selected spectral band:

$$\pi_{\mathcal{C}}(v_t) = \frac{1}{|\mathcal{C}|} \sum_{c \in \mathcal{C}} \langle v_t, V_{:,c} \rangle^2 . \tag{3}$$

Because ordinary steps dominate the unlabeled reference corpus, decisive-error steps exhibit smaller projection magnitudes on the selected directions than ordinary steps (Figure 3). We therefore define $S(s_t) = \frac{1}{\pi_{\mathcal{C}}(v_t)+\epsilon}$. so that a larger score indicates stronger decisive-error evidence. $\epsilon > 0$ is a small constant that ensures numerical stability. Note that R is constructed solely from the step-unlabeled reference collection where no decisive-error or responsible-agent annotation is used.

3.2 Dependency-aware rescoring

Attention-guided step dependencies. [Finn: add context cutoff explanation]appendix The base score in Eqn. 3 evaluates each step independently and therefore cannot distinguish an error source from its downstream consequences. Once a decisive error occurs, later steps may inherit the corrupted context and exhibit stronger failure signals than the original error. An earlier step is therefore more plausible as the root cause when subsequent anomalous steps depend strongly on it. To capture this structure, we construct a soft dependency graph over the target trajectory.

For every pair (i, t) with $i < t$, we define a non-negative coefficient $w_{i,t}$ that measures how strongly the later step s_t depends on the earlier step s_i. We derive this coefficient from the post-softmax attention produced by $\mathcal{M}$. Specifically, we aggregate the attention routed from tokens in s_t to tokens in s_i, yielding an attention mass $m_{i,t}$. We then normalize over all predecessor steps:

$$w_{i,t} = \frac{m_{i,t}}{\sum_{j<t} m_{j,t}}, \qquad i < t. \tag{4}$$

Thus, $\sum_{i<t} w_{i,t} = 1$, and $w_{i,t}$ represents the relative dependency of step s_t on predecessor s_i. The resulting weights provide soft evidence about which preceding steps contribute to the proxy model's representation of a later step. They require neither repeated LLM prompting, which introduces substantial inference cost (Zhang et al., 2025c; Yeh et al., 2026), nor explicit dependency reconstruction through LLM-based graph reasoning (Rafi et al., 2026; Wang et al., 2026), nor task-specific post-training on attributed failure trajectories (Zhang et al., 2025a;b).

Rescoring. Given the dependency weights, we treat the error signal of a downstream step as additional evidence for the earlier steps on which it depends. For each step s_i, we compute the dependency-weighted average of the base scores of its downstream dependents:

$$B_i = \frac{\sum_{t>i} w_{i,t}\, S(s_t)}{\sum_{t>i} w_{i,t}}, \qquad B_T = 0. \tag{5}$$

We then combine the step's local error signal with the downstream evidence attributed to it:

$$\widetilde{S}(s_i) = S(s_i) + \gamma B_i, \qquad \gamma \in [0, 1]. \tag{6}$$

Here, γ controls the strength of dependency-aware rescoring, and $\gamma = 0$ recovers the base score.

The normalization in Eqn. 5 produces a dependency-weighted average, which prevents an early or frequently attended step from receiving a large correction merely because it has many downstream steps. Instead, a step is promoted when the later steps that depend on it consistently exhibit strong failure signals. Conversely, an anomalous downstream step contributes little evidence to a predecessor on which it places little dependency weight.

Finally, we predict the decisive-error step and the corresponding responsible agent as

$$\hat{t} = \arg\max_{i \in \{1,\ldots,T\}} \widetilde{S}(s_i), \qquad \hat{a} = a_{\hat{t}}. \tag{7}$$

4 Experiments

4.1 Setup

Datasets. We evaluate our method Soap on 3 failure-attribution benchmarks comprising 5 subsets: Who&When (Zhang et al., 2025c), with algorithm-generated (WW-AG, 126 trajectories) and hand-crafted (WW-HC, 58) subsets; CORRECT-Error (Yu et al., 2025) (CE, 2,226); and TraceElephant (Chen et al., 2026), with Captain-Agent (TE-Cap, 85) and Magentic-One (TE-Mag, 91) subsets. They cover trajectories of varying lengths and diverse QA, coding, and tool-use tasks. Each trajectory is annotated with the decisive-error step and responsible agent. For each subset, we randomly split trajectories into unlabeled-reference, validation, and test sets using a 30/20/50% ratio. The reference annotations are hidden and used only to construct the spectral matrix R; validation labels are used for hyperparameter selection, and test labels only for final evaluation. Splitting is performed at the trajectory level. Full statistics and preprocessing details are provided in Appendix ??. [Finn: in appendix, add number of steps statistics]

Metrics. Following Zhang et al. (2025c), our primary metric is Step-Level Accuracy, the fraction of trajectories for which the predicted decisive step exactly matches the annotation. We additionally report Agent-Level Accuracy, Step@k, Mean Reciprocal Rank (MRR), and ± 1/ $\pm$ 2 tolerance accuracy in Appendix ??.

Baselines. We compare against two families, mirroring the row groups of Table 1: (i) prompt-based methods, which ask a judge LLM to name the decisive step — All-at-Once, Step-by-Step, and Binary Search (Zhang et al., 2025c), the schema-retrieval method CORRECT (Yu et al., 2025), the causal-graph method CHIEF (Wang et al., 2026), and the fault-reasoning judge RAFFLES (Zhu et al., 2026a); (ii) training-based methods, which, like Soap, score the trajectory's step representations — OAT (Yeh et al., 2026), which learns the latent dynamics of successful trajectories through one-class training, and StepFinder (Zhu et al., 2026b), a step classifier trained on synthetically corrupted trajectories with step-level error labels. Our main comparison (Table 1) uses the strict without-ground-truth (GT) setting, in which no method receives the gold task answer (Zhang et al., 2025c). For the with-GT setting we adapt Soap, which never otherwise consults the answer: the gold answer is appended to the task description in the proxy's context, and nothing else changes (Table 2)

Implementation details. We instantiate Soap with Qwen3.5-9B (Qwen Team, 2025) and DeepSeek-R1-Distill-Llama-8B (Guo et al., 2025). Step representations are obtained by mean-pooling token hidden states. We select the representation layer, the spectral band $\mathcal{C}$, the attention layer band, and the propagation strength γ on the validation set; §4.3 analyzes the sensitivity to each choice. The prompt-based methods use GPT-4o as the judge (GPT-5 in Appendix ??), evaluated on the same test splits as Soap. To ensure a fair comparison, the training-based baselines run on the same two backbones as Soap and are evaluated on identical test data, employing the default experimental configurations as outlined in their respective papers. Further implementation details appear in Appendices ?? and ??.

Table 1: Step-level accuracy (%) in the without-ground-truth setting (no method receives the gold task answer); mean over three seeds (std in Appendix ??). Prompt-based methods are evaluated with a closed-source judge; all other methods run on the two open backbones. Supervised: the method consumes step-level error annotations; the column is empty for prompt-based methods, which train nothing (CORRECT builds its schema library from annotated trajectories). The Soap row is our method; its spectral base score alone appears as the base-score row of Table 4. Best per column within each backbone block in bold, second best underlined. [Finn: Prompt rows: GPT-4o judge, evaluated on the same seed-split test sets as Soap; GPT-5 in Appendix.]

Method	Supervised	WW-AG	WW-HC	CE	TE-Cap	TE-Mag
Backbone: GPT-4o						
Prompt-based methods						
All-at-Once (Zhang et al., 2025c)		14.81	6.90	28.39	16.28	5.80
Step-by-Step (Zhang et al., 2025c)		20.63	13.79	44.47	13.95	13.04
Binary Search (Zhang et al., 2025c)		22.22	4.60	9.70	9.30	6.52
CORRECT (Yu et al., 2025)	✓	15.34	4.60	35.12	10.85	13.77
CHIEF (Wang et al., 2026)		25.93	14.94	38.56	13.95	8.70
RAFFLES (Zhu et al., 2026a)		35.98	18.39	53.05	23.26	23.91
Backbone: Qwen3.5-9B						
Training-based methods						
StepFinder (Zhu et al., 2026b)	✓	15.87	13.33	30.03	18.29	6.67
OAT (Yeh et al., 2026)	✗	16.72	10.11	56.50	15.35	18.84
Soap (ours)	✗	47.62	34.48	61.78	35.66	23.19
Backbone: DeepSeek-R1-Distill-Llama-8B						
Training-based methods						
StepFinder (Zhu et al., 2026b)	✓	18.94	10.80	36.91	14.73	4.20
OAT (Yeh et al., 2026)	✗	12.70	15.86	52.50	17.98	13.04
Soap (ours)	✗	45.50	28.74	64.68	42.64	30.43

4.2 Main Results

Comparison with competitive baselines. Table 1 compares Soap with both baseline families on the five subsets, strictly without the ground-truth answer; agent-level results for the same grid appear in Appendix ??. Notably, Soap is the only method constructed on the actual trajectories while requiring no step-level annotation — and one of its two strands is the outright best in every column. The margins are widest on Who&When: 47.62 vs. 35.98 for the strongest judge, RAFFLES, on WW-AG and 34.48 vs. 18.39 on WW-HC, even though Soap reads the trajectory through a frozen 9B proxy while the judges read it through GPT-4o. The training-based baselines confirm that the failure trajectories themselves carry the signal: OAT, which models only successful behavior, and StepFinder, whose supervision comes from a synthetic corpus, trail Soap in every cell, by as much as 31 points (16.72 and 15.87 vs. 47.62 on WW-AG). CE is the closest column — its trajectories are short, and OAT's one-class prior reaches 56.50 — yet Soap still leads with 61.78. The ordering repeats on DeepSeek-8B, which widens the lead on TraceElephant (42.64 on TE-Cap, 30.43 on TE-Mag), so the conclusions do not hinge on one proxy family.

Scalability to larger backbones. Figure 4 asks whether Soap keeps working, and keeps its margin over the training-based baselines, as the proxy grows. Every method reads the same backbone — Qwen3-14B and Qwen3.5-27B; the Qwen3.5-9B point is Table 1 — on WW-AG and WW-HC, with the splits of Table 1; Soap is re-selected at each size, OAT and StepFinder are trained at each size and averaged over five training seeds. Soap holds its level with scale rather than growing with it: within the Qwen3.5 family the 27B point matches the 9B point on both subsets (44.97 vs. 47.62 on WW-AG, 34.48 on both on WW-HC), and its selected configuration mirrors the 9B one — a late layer and, on WW-AG, a band that skips the top component. Rescoring lifts every backbone on WW-AG (+2.6 to +8.5) and the two Qwen3.5 sizes on WW-HC. The 14B point crosses to the previous generation (Qwen3, dense) and is the one dip: competitive on WW-AG (42.86) but ten points lower on WW-HC, where rescoring finds nothing to correct; the clean comparison is 9B → 27B

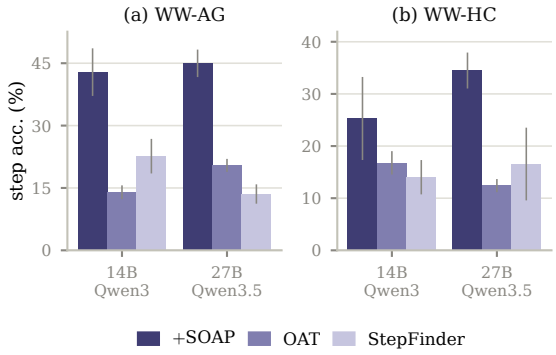


Figure 4: Scalability to larger backbones. Step-level accuracy (%) on (a) WW-AG and (b) WW-HC for two proxies larger than Table 1's Qwen3.5-9B: Qwen3-14B (the previous generation; no Qwen3.5 model of that size exists) and Qwen3.5-27B. Bars, dark to light: Soap, OAT, StepFinder; error bars ±1 std (over seeds for Soap; over training seeds for the baselines, each at the mean over seeds). Splits and protocol as in Table 1.

within Qwen3.5. Neither baseline scales: OAT stays at 14–20 on WW-AG and 10–17 on WW-HC, StepFinder at 13–23 and 11–17 with no monotone trend, so the margin is intact at every size (the best baseline cell, 22.65, against Soap's worst, 42.86, on WW-AG).

Does Soap generalize across data distributions? Figure 5(a) probes whether a fitted Soap is specific to the distribution it was tuned on. For each source–target pair among {WW-AG, WW-HC, TE-Cap, TE-Mag}, we fit the SVD reference on the source's train split, then the full configuration is re-selected. Dependency weights are always computed from the target trajectories' own attention, and seeds pair positionally (source seed i's reference with target seed i's splits). The diagonal repeats the in-domain protocol of Table 1 verbatim. Backbone: Qwen3.5-9B (DeepSeek-8B in Appendix ??).

The fitted reference travels better than one might expect once the hyperparameters are re-tuned. The in-domain pair still leads every column, but by a wide margin only on WW-AG (47.62 in-domain vs. 35.45 for the best foreign source); on the other three targets the best foreign reference lands within about one point of in-domain (33.33 vs. 34.48 on WW-HC, 34.88 vs. 35.66 on TE-Cap, 22.46 vs. 23.19 on TE-Mag), and on DeepSeek a WW-HC-fitted reference even matches TE-Cap's in-domain 42.64. In contrast, freezing the source's configuration collapses transfer (WW-AG→WW-HC falls to 3.45): the distribution-specific part of Soap is chiefly the hyperparameter configuration, while the spectral reference itself is broadly reusable — so adapting to a new system needs target-side tuning data more than it needs target-side reference trajectories.

Results in the with-GT setting. Table 2 repeats the comparison when the gold task answer is available, the setting in which prompt-based judges are strongest (Zhang et al., 2025c). Even here Soap leads: 43.39 vs. 34.39 for the best judge (RAFFLES) on WW-AG, and 34.48 vs. 21.84 on WW-HC. Its adaptation could hardly be smaller — the answer enters the proxy's context and nothing else changes — yet a frozen 9B proxy still localizes the decisive step more reliably than a GPT-4o judge that holds the answer. Among the training-based baselines OAT profits most from the appended answer (22.12 on WW-AG); every baseline remains at least nine points behind.

Results with synthetic trajectories. The reference R requires only unlabeled trajectories, which raises the fully self-contained scenario in which no corpus from the target system is available for fitting. Figure 5(b) evaluates this scenario on WW-AG and WW-HC: we replace the train-split trajectories with synthetic ones generated by a multi-agent harness and fit R on them, keeping the validation and test splits of Table 1 unchanged. We test two generators: (1) the proxy model itself, Qwen3.5-9B, and (2) a stronger model, GPT-4o, probing whether higher-quality generation yields better accuracy. Unlike the benchmark corpora, which contain failures only, the generated trajectories are used as-is, regardless of task success.

Table 2: Step-level accuracy (%) in the with-ground-truth setting on Who&When, where the gold task answer is provided; Soap is adapted to this setting as described in §4.1. Best per column within each backbone block in bold, second best underlined.

Method	Sup.	WW-AG	WW-HC
Backbone: GPT-4o			
Prompt-based methods			
Step-by-Step		22.75	<u>18.39</u>
CORRECT	✓	15.87	4.60
CHIEF		<u>24.34</u>	11.49
RAFFLES		34.39	21.84
Backbone: Qwen3.5-9B			
Training-based methods			
StepFinder	✓	18.52	<u>12.87</u>
OAT	✗	<u>22.12</u>	8.05
Soap (ours)	✗	43.39	34.48

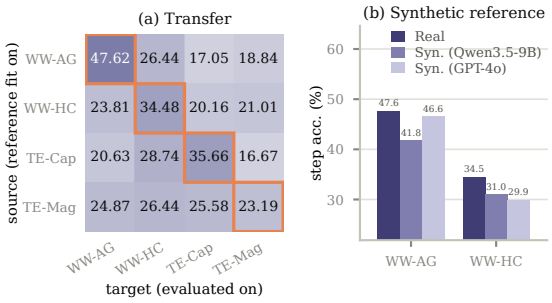


Figure 5: (a) Cross-distribution transfer (source rows, target columns; outlined diagonal = in-distribution, equals Soap in Table 1) and (b) synthetic reference corpora: Soap step-level accuracy (%) with the SVD reference fit on the real train split versus on trajectories generated by Qwen3.5-9B or GPT-4o; validation and test splits as in Table 1. Backbone Qwen3.5-9B.

Generation details are in Appendix ??. A synthetic reference nearly suffices. Fitting R on the proxy's own generations costs 5.8 points on WW-AG (41.80 vs. 47.62) and 3.5 on WW-HC; the GPT-4o corpus closes WW-AG to within 1.1 points (46.56). Rescoring keeps its lift on every synthetic reference, and every synthetic cell stays far above the training-based baselines of Table 1. Neither generator dominates — GPT-4o wins WW-AG, the open-weight Qwen3.5-9B corpus wins WW-HC — so a system's own cheap re-runs, unlabeled and unfiltered, can stand in for a reference corpus that does not exist.

4.3 Analysis

The analyses in this section ablate the individual components of Soap, all under the same protocol: starting from the validation-selected configuration of Table 1 (Appendix ??), we vary exactly one axis and hold every other hyperparameter fixed; the anchor's own value therefore appears as one point of every sweep and coincides with the main-table result. Unless stated otherwise, ablations use backbone Qwen3.5-9B on WW-AG and WW-HC; DeepSeek-8B mirrors are in Appendix ??.

Comparison with other scoring functions. Table 3 replaces the spectral base score with seven alternative per-step scoring functions: (1) perplexity, the mean negative log-likelihood of the step's tokens under $\mathcal{M}$, in context; (2) random subspace, the projection onto $|\mathcal{C}|$ random singular directions; (3) top subspace, the projection onto the leading $|\mathcal{C}|$ singular directions; (4) tail subspace, the projection onto the trailing $|\mathcal{C}|$ directions; (5) full spectrum, the projection onto all computed singular directions; and (6–7) the ℓ_1 and ℓ_2 norms of the step representation.

Table 3: Alternative scoring functions. Step-level accuracy (%) with no rescoring, Qwen3.5-9B.

Scoring function	WW-AG	WW-HC
Perplexity	3.70	8.05
Random subspace	13.23	9.20
Top subspace	12.70	33.33
Tail subspace	6.35	6.90
Full spectrum	12.17	31.03
Norm ℓ_1	18.52	18.39
Norm ℓ_2	14.29	26.44
Spectral band (ours)	39.15	33.33

Table 4: Alternatives to attention-guided rescoring. Step-level accuracy (%), Qwen3.5-9B; all rows share the spectral base score. Weight rows replace only the dependency weights; position rows replace the whole correction (λ selected per column from $\{0.1, \ldots, 1.0\}$). The last row equals Soap in Table 1. Best in bold.

Rescoring design	WW-AG	WW-HC	CE	TE-Cap	TE-Mag
Base score (no rescoring)	39.15	33.33	61.38	33.33	21.01
Content-agnostic weights					
Uniform, unnormalized	16.93	16.09	57.85	14.73	5.07
Uniform, normalized	40.21	34.48	60.78	35.66	15.94
Position heuristics					
Temporal bias, z-scored	40.74	34.48	61.62	34.11	21.01
Temporal bias, raw	22.22	3.45	41.65	6.20	5.07
Earliest of top-5	21.16	14.94	4.51	9.30	5.07
Attention-guided (ours)	47.62	34.48	61.78	35.66	23.19

The spectral band wins or ties every cell. The discriminating column is WW-AG, where the selected band starts at component 1: dropping the top singular direction is worth 26 points over the top subspace (39.15 vs. 12.70) and over the full spectrum (12.17) — where the band sits matters, not merely its width. On WW-HC the selected band starts at component 0, so the top subspace coincides with ours by construction. Perplexity, the random and tail subspaces, and both norms trail far behind everywhere: the anomaly is not loud in likelihood or vector magnitude, but in a small displacement off the common mode.

Effect of attention-guided rescoring. Table 4 asks whether attention-guided rescoring has meaningful advantages over base scores and other simpler rescoring techniques. Keeping the base score and γ fixed, we replace the weights in Eqn. 5 with two content-agnostic alternatives: uniform (normalized), which averages the scores of all successors with equal weight, and uniform (unnormalized), which sets every weight to one without normalization, so each step receives the raw sum of its successors' scores and earlier steps aggregate more terms. Additionally, we also apply temporal bias, which adds $-\lambda t/T$ to the score of step t in a z-scored variant (base score standardized within each trajectory first) and a raw variant, with $\lambda \in \{0.1, \ldots, 1.0\}$ selected as for γ, and earliest of top-k, which predicts the earliest step among the k=5 highest-scoring steps. The unnormalized sum is destructive: because earlier steps aggregate more terms, it collapses toward predicting the first step and falls far below the base score everywhere (16.93 vs. 39.15 on WW-AG). The normalized mean repairs the length artifact but not the attribution: it hovers around the base score and never reaches Soap where the correction matters most (40.21 vs. 47.62 on WW-AG, 15.94 vs. 23.19 on TE-Mag). The position heuristics fare no better. The z-scored temporal bias buys at most a point or two over the base score — an early-step prior does carry a little signal — but stays seven points short of Soap on WW-AG; the raw bias is degenerate, since the inverse-projection score spans orders of magnitude and a bounded bias either vanishes against it or overwhelms it (3.45 on WW-HC); hard-committing to the earliest of the top five is worse than the base score alone (4.51 on CE). Soap promotes an early step only when its successors' anomaly points at it: the gain comes from which successors the attention weights select and how strongly they blame each predecessor, not from aggregation or position as such.

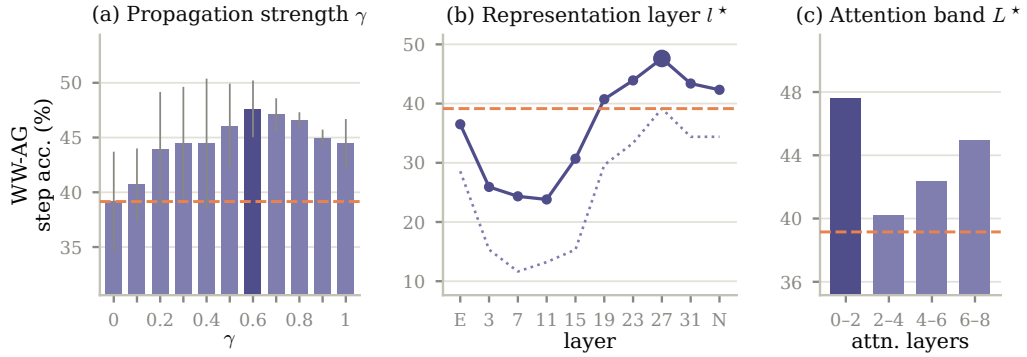


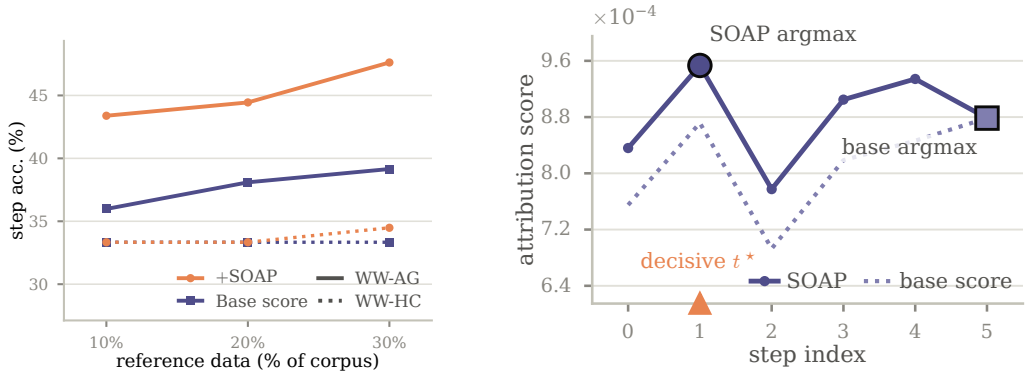
Figure 6: Sensitivity of Soap. Step-level accuracy (%) on WW-AG, backbone Qwen3.5-9B, varying one axis at a time from the selected configuration of Table 1: (a) propagation strength γ (error bars: ± 1 std over seeds), (b) the layer $l^\star$ whose hidden states are pooled into step representations (E: embeddings; N: final normed states), and (c) the attention layer band $L^\star$ that builds the dependency weights. Light bars (and, in (b), the solid line) show Soap at the varied value; the dark bar or large marker is the selected configuration; the dotted line in (b) is the base score at each layer; the dashed orange line is the base score of the selected configuration, i.e. the spectral score with no rescoring (γ=0). WW-HC, TraceElephant, and DeepSeek-8B in Appendix ??.

Sensitivity to the hyperparameters. Figure 6 varies three axes one at a time, on WW-AG; in every panel the dashed line is the base score of the selected configuration (no rescoring) and the dark mark is the selected configuration. WW-HC, TraceElephant, and DeepSeek-8B repeat the sweep in Appendix ??. The context window w of the dependency weights is ablated in Appendix ?? (Table ??). Propagation strength γ (a) reveals two regimes: on WW-AG accuracy climbs with γ and peaks at 0.6 (47.62), so the correction carries much of the signal; on WW-HC it is a nudge — a small peak at γ=0.1, then a steep decay as the propagated blame drowns the base score (8.05 at γ=1.0). The intermediate points interpolate smoothly, so the operating range is wide within each regime. Representation layer $l^\star$ (b) shows that the signal lives late in the stack: on WW-AG accuracy rises through the depth and peaks at layer 27 of 31; on WW-HC it concentrates almost entirely in the final layer (33.33 at layer 31 vs. at most 19.54 mid-stack) — the anomaly there is a property of the model's final summary of the step, not of any intermediate computation. Rescoring lifts nearly every layer on WW-AG, embedding included (28.57 to 36.51), so the correction is not tuned to one layer's quirks. Attention band $L^\star$ (c), the layers whose attention builds the dependency weights, matters least: it shifts accuracy by a few points where γ is large (40.21–47.62 across bands on WW-AG) and is flat where the correction is a nudge.

Quantity of unlabeled reference data. Because the reference is fit on unlabeled trajectories, Figure 7(a) characterizes data efficiency: with the validation (20%) and test (50%) splits fixed as in the main setting, we fit R on random subsamples of the train split covering $\{10, 20, 30\}$% of the corpus and re-select the full configuration at each fraction, then report base and rescored accuracy. Soap needs remarkably little reference data. At a third of the corpus — twelve trajectories on WW-AG, six on WW-HC — every cell sits within a few points of its full-data accuracy (43.39 vs. 47.62 on WW-AG; 33.33 vs. 34.48 on WW-HC), and the gain of rescoring over the base score is already established. The spectral common mode is low-rank enough that a handful of trajectories pins it down; more data mostly sharpens the band selection.

Qualitative example. Figure 7(b) shows a TE-Cap trajectory on which rescoring flips the prediction from wrong to right. The base score peaks late, on an Orchestrator turn that only summarizes what earlier steps produced; the dependency weights route that late anomaly back to the step the later turns built on: the decisive error is the extraction agent's malformed Unlambda program (a missing backtick), four steps before the base score's argmax.

efficiency comparison to be in the appendix.



(a) (b)

Figure 7: (a) Quantity of unlabeled reference data: step-level accuracy (%) with the reference fit on $\{10, 20, 30\}$% of the corpus, before (S, dotted) and after (+Soap, solid) rescoring; backbone Qwen3.5-9B. (b) Rescoring recovers the source: per-step scores on a TE-Cap test trajectory, backbone Qwen3.5-9B, at the selected configuration of Table 1: the base score (dotted) peaks on a late step that inherited the corrupted context; Soap (solid) lifts the decisive error $t^\star$ (orange marker on the axis) to the top. The large square and circle mark the base score's and Soap's argmax.

5 Related Work

Agentic failure attribution has gained interest recently for ensuring LLM-based agents' reliability (Qi et al., 2026; Barke et al., 2026). Zhang et al. (2025c) formulate automated failure attribution, introduce the Who&When benchmark, and evaluate All-at-Once, Step-by-Step, and Binary Search LLM judges; complementary work categorizes common multi-agent failure modes (Cemri et al., 2026; Deshpande et al., 2025; Raj et al., 2026). Subsequent methods either improve inference-time prompting (Banerjee et al., 2025; Zhu et al., 2026a; Yu et al., 2025) or train dedicated attributors. For instance, AgenTracer and StepFinder instead learn from labeled or synthetically corrupted trajectories using reinforcement learning or temporal-semantic modeling (Zhang et al., 2025a; Zhu et al., 2026b). These approaches, however, often rely on computationally intensive inference or large-scale step-level error annotations, limiting their practicality for real-world deployment. A recent approach OAT mitigates this by learning continuous latent dynamics from clean successful trajectories by one-class learning (Yeh et al., 2026), which provides no decisive error signals in their data unlike our adopted step-unlabeled failure trajectories thus underperforms our Soap (Table ??). Several methods model cross-step dependencies (Rafi et al., 2026). GraphTracer derives information-flow graphs from references to prior outputs and learns to trace root causes and propagation paths (Zhang et al., 2025b); CHIEF construct causal dependency graphs during prompting through multi-stage LLM reasoning (Wang et al., 2026). We compare with both of them in Table ??. Liu et al. (2026) use attention signals for failure attribution but different from Soap, they calculate attention scores on steps with high negative log-likelihood for two-stage localization.

6 Conclusion

AI use statement

(This section is required and does not count toward the page limit.)

In this work, we used generative AI tools for [tasks with required disclosure]. We have not used generative AI tools for [other tasks with required disclosure], and [the rest of the required disclosure tasks] are not applicable to this work. Additionally, we used generative AI tools for [tasks with recommended disclosure]. We have reviewed all AI-assisted work. [Elaborate. For example, "we checked LLM-generated research ideas for potential plagia-

rism through a manual literature survey", "LLM-generated code was verified and tested for correctness by 2 authors", etc.]. We take responsibility for the final content of this work, including text, claims or artifacts produced with the aid of generative AI.

See the ICLR 2027 AI Policy for Authors for more details. This statement should not be more than 1 page.

Ethics statement

(This section is recommended and does not count toward the page limit.)

If authors feel that their paper submission raises questions regarding the Code of Ethics, they are encouraged to include a paragraph of Ethics Statement (at the end of the main text before references) to address potential concerns where appropriate. Topics include, but are not limited to, studies that involve human subjects, practices to data set releases, potentially harmful insights, methodologies and applications, potential conflicts of interest and sponsorship, discrimination/bias/fairness concerns, privacy and security issues, legal compliance, and research integrity issues (e.g., IRB, documentation, research ethics). This statement should not be more than 1 page.

Reproducibility statement

(This section is recommended and does not count toward the page limit.)

It is important that the work published in ICLR is reproducible. Authors are strongly encouraged to include a paragraph-long Reproducibility Statement at the end of the main text (before references) to discuss the efforts that have been made to ensure reproducibility. This paragraph should not itself describe details needed for reproducing the results, but rather reference the parts of the main paper, appendix, and supplemental materials that will help with reproducibility. For example, for novel models or algorithms, a link to an anonymous downloadable source code can be submitted as supplementary materials; for theoretical results, clear explanations of any assumptions and a complete proof of the claims can be included in the appendix; for any datasets used in the experiments, a complete description of the data processing steps can be provided in the supplementary materials. Each of the above are examples of things that can be referenced in the reproducibility statement.

References

Adi Banerjee et al. Where did it all go wrong? a hierarchical look into multi-agent error attribution. arXiv preprint arXiv:2510.04886, 2025.

Shraddha Barke, Arnav Goyal, Alind Khare, Avaljot Singh, Suman Nath, and Chetan Bansal. AgentRx: Diagnosing AI agent failures from execution trajectories. arXiv preprint arXiv:2602.02475, 2026.

Mert Cemri, Melissa Z Pan, Shuyi Yang, Lakshya A Agrawal, Bhavya Chopra, Rishabh Tiwari, Kurt Keutzer, Aditya Parameswaran, Dan Klein, Kannan Ramchandran, Matei Zaharia, Joseph E. Gonzalez, and Ion Stoica. Why do multi-agent LLM systems fail? In The Thirty-ninth Annual Conference on Neural Information Processing Systems Datasets and Benchmarks Track, 2026. URL https://openreview.net/forum?id=fAjbYBmonr.

Mengzhuo Chen, Junjie Wang, Fangwen Mu, Yawen Wang, Zhe Liu, Huanxiang Feng, and Qing Wang. Seeing the whole elephant: A benchmark for failure attribution in LLM-based multi-agent systems. In Maria Liakata, Viviane P. Moreira, Jiajun Zhang, and David Jurgens (eds.), Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), pp. 19888–19905, San Diego, California, United States, July 2026. Association for Computational Linguistics. ISBN 979-8-89176-390-6. doi: 10.18653/v1/2026.acl-long.912. URL https://aclanthology.org/2026.acl-long.912/.

Darshan Deshpande, Varun Gangal, Hersh Mehta, Jitin Krishnan, Anand Kannappan, and Rebecca Qian. Trail: Trace reasoning and agentic issue localization. arXiv preprint arXiv:2505.08638, 2025.

Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Ruoyu Zhang, Runxin Xu, et al. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. arXiv preprint arXiv:2501.12948, 2025.

Peter J. Huber. Robust estimation of a location parameter. The Annals of Mathematical Statistics, 35(1):73–101, 1964. doi: 10.1214/aoms/1177703732.

Yang Liu, Hongjiang Feng, Junsong Pu, and Zhuangbin Chen. MASPrism: Lightweight failure attribution for multi-agent systems using prefill-stage signals. arXiv preprint arXiv:2605.07509, 2026.

Yunjia Qi, Zehua Yin, Xintong Shi, Hao Peng, Songyuanyi Lu, Yixian Liu, Richeng Xuan, Yuhong Liu, Zhichao Hu, Xiaozhi Wang, Lei Hou, Bin Xu, and Juanzi Li. TRAJDEBUG: Tracing error lifecycle to identify critical failures in long-horizon agent trajectories. arXiv preprint arXiv:2608.06346, 2026.

Qwen Team. Qwen3 technical report. arXiv preprint arXiv:2505.09388, 2025.

Md Nakhla Rafi, Md Ahasanuzzaman, Dong Jae Kim, Zhijie Wang, and Tse-Hsun Chen. Falat: Tracing failures in llm agent trajectories via dependency-guided search, 2026. URL https://arxiv.org/abs/2606.00765.

Harsh Raj, Vipul Gupta, Anas Mahmoud, Razvan-Gabriel Dumitru, Darvin Yi, Aakash Sabharwal, and Yunzhong He. Model or harness? an interaction-centric taxonomy for localizing agent failures. arXiv preprint arXiv:2607.28802, 2026.

Lei Wang, Chen Ma, Xueyang Feng, Zeyu Zhang, Hao Yang, Jingsen Zhang, Zhiyuan Chen, Jiakai Tang, Xu Chen, Yankai Lin, Wayne Xin Zhao, Zhewei Wei, and Jirong Wen. A survey on large language model based autonomous agents. Frontiers of Computer Science, 18(6):186345, 2024. doi: 10.1007/s11704-024-40231-1.

Yawen Wang, Wenjie Wu, Junjie Wang, and Qing Wang. From flat logs to causal graphs: Hierarchical failure attribution for llm-based multi-agent systems, 2026. URL https://arxiv.org/abs/2602.23701.

Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik R. Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In International Conference on Learning Representations, 2023. URL https://openreview.net/forum?id=WE_vluYUL-X.

Samuel Yeh, Yiwen Zhu, Shaleen Deep, and Sharon Li. Tracing agentic failure from the flow of success. arXiv preprint arXiv:2607.12747, 2026.

Yifan Yu, Moyan Li, Shaoyuan Xu, Jinmiao Fu, Xinhai Hou, Fan Lai, and Bryan Wang. CORRECT: COndensed eRror RECognition via knowledge transfer in multi-agent systems. arXiv preprint arXiv:2509.24088, 2025.

Guibin Zhang, Junhao Wang, Junjie Chen, Wangchunshu Zhou, Kun Wang, and Shuicheng Yan. AgenTracer: Who is inducing failure in the llm agentic systems? arXiv preprint arXiv:2509.03312, 2025a.

Heng Zhang, Yuling Shi, Xiaodong Gu, Haochen You, Zijian Zhang, Lubin Gan, Yilei Yuan, and Jin Huang. Graphtracer: Graph-guided failure tracing in llm agents for robust multi-turn deep search, 2025b. URL https://arxiv.org/abs/2510.10581.

Shaokun Zhang, Ming Yin, Jieyu Zhang, Jiale Liu, Zhiguang Han, Jingyang Zhang, Beibin Li, Chi Wang, Huazheng Wang, Yiran Chen, and Qingyun Wu. Which agent causes task failures and when? on automated failure attribution of llm multi-agent systems. In International Conference on Machine Learning (ICML), 2025c.

Chenyang Zhu, Spencer Hong, Jingyu Wu, Kushal Chawla, Yuhui Tang, Youbing Yin, Nathan Wolfe, Erin Babinsky, and Daben Liu. RAFFLES: Reasoning-based attribution of faults for LLM systems. In Vera Demberg, Kentaro Inui, and Lluís Marquez (eds.), Proceedings of the 19th Conference of the European Chapter of the Association for Computational Linguistics (Volume 1: Long Papers), pp. 7659–7688, Rabat, Morocco, March 2026a. Association for Computational Linguistics. ISBN 979-8-89176-380-7. doi: 10.18653/v1/2026.eacl-long.359. URL https://aclanthology.org/2026.eacl-long.359/.

Taiyu Zhu, Yifan Wu, Weilin Jin, Ying Li, and Gang Huang. StepFinder: A temporal semantic framework for failure attribution in multi-agent systems. In Proceedings of the 32nd ACM SIGKDD Conference on Knowledge Discovery and Data Mining V. 2, 2026b.